UNIT - I : PYTHON BASICS AND CONTROL FLOW

1. History of Python

* Created by GUIDO VAN ROSSUM at CWI (Netherlands) in the late 1980s (started Dec 1989), first released Feb 1991 (version 0.9.0). Named after the comedy show 'Monty Python's Flying Circus', NOT the snake.
* Python 1.0 - 1994. Python 2.0 - 2000 (list comprehension, garbage collection). Python 3.0 - December 2008 (not backward compatible). Python 2 reached end of life on 1 January 2020. Managed by the Python Software Foundation (PSF).
* Influenced by ABC, C, Modula-3, Lisp, Haskell, Java. Features: simple and readable syntax, interpreted, high-level, dynamically typed, object oriented, free and open source, portable (platform independent), extensible, huge standard library ('batteries included'), automatic memory management.

Thrust areas of Python

* Web development (Django, Flask), Data science and analytics (NumPy, Pandas, Matplotlib), Machine learning / AI (scikit-learn, TensorFlow, PyTorch), Scientific computing (SciPy), Automation and scripting, Game development (Pygame), Desktop GUI (Tkinter), Networking / cyber security, Embedded and IoT (Raspberry Pi, MicroPython), Web scraping, Database access, Education.

2. Installing Anaconda and Jupyter Notebook

* ANACONDA = free distribution of Python and R for data science. It includes Python, 250+ packages (NumPy, Pandas, Matplotlib, SciPy), the conda package manager, Jupyter Notebook, Spyder IDE. Available for Windows, macOS, Linux.
* Steps: (1) Go to anaconda.com and download the installer for your OS

(64-bit, Python 3.x). (2) Run the installer, accept the licence, choose 'Just Me', select install location. (3) Finish; open 'Anaconda Navigator' or 'Anaconda Prompt'. (4) Check with:

```
> conda --version
> python --version
> conda install numpy        # install a package
> conda list                 # show installed packages
```

Jupyter Notebook

* Web based interactive environment: a notebook (.ipynb) holds CODE cells, MARKDOWN cells, output, graphs. Launch from Anaconda Navigator -> Launch Jupyter Notebook, or type 'jupyter notebook' at the prompt; opens in browser at localhost:8888.
* Use: New -> Python 3 notebook; type code in a cell; run with SHIFT + ENTER (CTRL + ENTER runs without moving); A / B insert cell above / below; D D delete cell; M markdown; Y code. Save as .ipynb; File -> Download as .py.

3. Parts of Python Programming Language

Identifiers

* Names given to variables, functions, classes, modules. Rules: start with a letter (A-Z, a-z) or underscore; followed by letters, digits, underscores; CASE SENSITIVE (age and Age are different); cannot be a keyword; no spaces or special symbols (@, $, %).

```
valid:   name, _count, total2, Student_Name
invalid: 2cost, my-name, class, for, a b
```

Keywords

* Reserved words with special meaning; cannot be used as identifiers. Python

3 has 35 keywords (changes with version): False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield. Only True, False, None start with capital.

```
import keyword
print(keyword.kwlist)
print(len(keyword.kwlist))
```

Statements and Expressions

* STATEMENT = an instruction that the interpreter executes (assignment, if, for, print, import). EXPRESSION = a combination of values, variables and operators that evaluates to a single VALUE (3 + 4 * 2). Every expression is part of a statement.
* A statement normally ends at the end of the line. Use backslash (\) or brackets for a multi-line statement; use semicolon (;) to put more than one statement on a line.

```
x = 10 + 5              # assignment statement, expr 10+5
total = 1 + 2 + \
        3 + 4           # line continuation
a = 1; b = 2            # two statements in one line
```

Variables

* A variable is a NAME that refers to an object (value) stored in memory. No declaration needed; type is decided by the value assigned (dynamic typing). Assignment operator is '='.

```
x = 5              # int
name = "Ravi"      # str
a = b = c = 0      # multiple assignment
p, q, r = 1, 2.5, "hi"   # multiple values
print(id(x))       # memory address of object
```

Operators

Type	Operators	Example
Arithmetic	+ - * / // % **	7//2 = 3, 7%2 = 1, 2**3 = 8, 7/2 = 3.5
Relational (comparison)	== != > < >= <=	5 > 3 is True
Assignment	= += -= *= /= //= %= **= &= |= ^= >>= <<=	x += 2 means x = x + 2
Logical	and or not	(5>3) and (2>1) is True
Bitwise	& | ^ ~ << >>	5 & 3 = 1, 5 | 3 = 7, 5 ^ 3 = 6, ~5 = -6, 5<<1 = 10
Membership	in not in	'a' in 'cat' is True
Identity	is is not	x is y (same object?)
Ternary (conditional)	value1 if condition else value2	big = a if a > b else b

Precedence and associativity

* PRECEDENCE decides which operator is evaluated first; ASSOCIATIVITY decides the order when operators have same precedence (mostly LEFT to right; ** and assignment are RIGHT to left).

Order (high -> low)	Operator
1	( ) parentheses
2	** exponent (right to left)
3	+x -x ~x unary
4	* / // %
5	+ -
6	<< >>
7	& then ^ then |
8	== != > < >= <= is is not in not in
9	not then and then or
10	if - else (ternary), = (assignment)

```
>>> 2 + 3 * 4          # * first
14
>>> 2 ** 3 ** 2        # right to left: 2 ** 9
512
>>> 100 / 10 * 2       # left to right
20.0
>>> not 1 > 2 and 3 < 5
True
```

Data types

* Everything is an object. Built-in types: NUMERIC: int (unlimited size), float (64-bit, 15 digits), complex (3+4j); BOOLEAN: bool (True / False); TEXT: str; SEQUENCE: list, tuple, range; MAPPING: dict; SET: set, frozenset; BINARY: bytes, bytearray; NONE: NoneType (None).

Type	Example	Mutable?
int	10, -45, 0b101, 0o17, 0xFF	No
float	3.14, 2.5e3	No
complex	2+3j (z.real, z.imag)	No
bool	True, False (= 1, 0)	No
str	'hello', "hi", '''multi'''	No
list	[1, 'a', 2.5]	Yes
tuple	(1, 2, 3)	No
set	{1, 2, 3}	Yes
dict	{'a': 1, 'b': 2}	Yes

Indentation

* Python uses INDENTATION (whitespace at start of a line) to define blocks - there are no braces { }. Standard is 4 spaces. All statements in a block must be indented equally; wrong indentation gives IndentationError. Block header ends with a colon ':'.

```
if 5 > 2:
    print("five is greater")    # inside block
    print("done")
print("outside block")
```

Comments

* Single line comment starts with # (ignored by the interpreter). Multi-line comment: use several # lines or triple quoted string (''' ... ''' - actually a docstring when first in a function / class).

```
# this is a comment
x = 5   # inline comment
'''
multi line
comment
'''
```

Reading input and printing output

* input(prompt) reads a line from keyboard and ALWAYS returns a STRING - convert with int(), float(). print() displays output; parameters sep (separator, default space), end (default newline), file.

```
name = input("Enter name: ")
age = int(input("Enter age: "))
print("Hello", name, "you are", age)
print("a", "b", "c", sep="-")        # a-b-c
print("Hi", end=" ")                 # no newline
print("there")                       # Hi there
print(f"{name} is {age} years old")  # f-string
print("%s is %d" % (name, age))      # % formatting
print("{} is {}".format(name, age))  # format()
```

Type conversions, type() and is

* IMPLICIT conversion: Python converts automatically (int + float -> float). EXPLICIT (type casting): int(), float(), str(), bool(), list(), tuple(), set(), complex(), chr(), ord().

```
>>> int("25") + 5
30
>>> float(7)
7.0
>>> int(3.99)            # truncates
3
>>> str(10) + "5"
'105'
>>> bool(0), bool("a")
(False, True)
>>> 5 + 2.0              # implicit -> float
7.0
>>> type(5), type(5.0), type("5"), type([1])
(<class 'int'>, <class 'float'>, <class 'str'>, <class 'list'>)
>>> a = [1]; b = a; c = [1]
>>> a is b, a is c, a == c
(True, False, True)
```

'==' compares VALUES; 'is' compares IDENTITY (same object in memory, id(a) == id(b)).

Dynamic and strongly typed language

* DYNAMICALLY typed: type of a variable is checked at RUN time and the same name can refer to different types: x = 5 then x = 'five' is allowed (no type declaration).
* STRONGLY typed: Python does NOT implicitly mix incompatible types - '5' + 3 raises TypeError; you must convert explicitly: int('5') + 3 = 8. (Weakly typed languages like JavaScript would give '53').

```
x = 10
print(type(x))       # <class 'int'>
x = "ten"
print(type(x))       # <class 'str'>
print("5" + 3)       # TypeError: can only concatenate str
```

4. Control Flow Statements

if statement

```
age = 20
if age >= 18:
    print("Eligible to vote")
```

if - else

```
n = int(input("Number: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

if ... elif ... else (ladder)

```
marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Fail"
print(grade)            # B
```

Nested if

```
a, b, c = 5, 9, 7
if a > b:
    if a > c:
        print("a is largest")
    else:
        print("c is largest")
else:
    if b > c:
        print("b is largest")
    else:
        print("c is largest")
```

while loop

* Repeats a block as long as the condition is True. Optional else: runs when loop ends normally (not by break). Beware of infinite loops.

```
i = 1
while i <= 5:
    print(i, end=" ")
    i += 1              # 1 2 3 4 5
else:
    print("loop finished")
```

for loop and range()

* for iterates over a sequence (list, string, tuple, dict, range). range(stop), range(start, stop), range(start, stop, step) gives numbers from start up to stop-1.

```
for i in range(5):              # 0 1 2 3 4
    print(i, end=" ")
for i in range(2, 10, 3):       # 2 5 8
    print(i, end=" ")
for ch in "abc":
    print(ch)
for i in range(10, 0, -2):      # 10 8 6 4 2
    print(i, end=" ")
```

break, continue and pass

* break - terminates the loop immediately. continue - skips the rest of the current iteration and goes to the next. pass - null statement, does nothing (placeholder).

```
for i in range(1, 10):
    if i == 3:
        continue       # skip 3
    if i == 6:
        break          # stop at 6
    print(i, end=" ")  # 1 2 4 5
```

Catching exceptions - try and except

* An EXCEPTION is a run-time error (ZeroDivisionError, ValueError, TypeError, IndexError, KeyError, FileNotFoundError, NameError). try block holds risky code; except handles it, so the program does not crash. else runs if no exception; finally ALWAYS runs (cleanup). raise throws an exception.

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Enter integers only")
except Exception as e:
    print("Error:", e)
else:
    print("No error")
finally:
    print("Done")
```

5. Sample Experiments (1 - 6)

Exp 1: Largest of three numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a >= b and a >= c:
    large = a
elif b >= a and b >= c:
    large = b
else:
    large = c
print("Largest number is", large)
# Output: inputs 12 45 30 -> Largest number is 45
```

Exp 2: Prime numbers within an interval

```
lo = int(input("Lower limit: "))
hi = int(input("Upper limit: "))
print("Primes between", lo, "and", hi)
for num in range(lo, hi + 1):
    if num > 1:
        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                break
        else:                   # no divisor found
            print(num, end=" ")
# Output (10 to 30): 11 13 17 19 23 29
```

Exp 3: Swap two numbers without temporary variable

```
a = int(input("a = "))
b = int(input("b = "))
print("Before swap:", a, b)
a = a + b
b = a - b
a = a - b
print("After swap:", a, b)
# other ways:  a, b = b, a      or   a = a ^ b; b = a ^ b; a = a ^ b
# Output: a=5, b=9 -> After swap: 9 5
```

Exp 4: Demonstration of operators

```
a, b = 10, 3
print("Arithmetic:", a+b, a-b, a*b, a/b, a//b, a%b, a**b)
print("Relational:", a>b, a<b, a==b, a!=b, a>=b, a<=b)
x = 10
x += 5; print("Assignment +=:", x)     # 15
x *= 2; print("Assignment *=:", x)     # 30
print("Logical:", a>5 and b<5, a>5 or b>5, not(a>5))
print("Bitwise:", a&b, a|b, a^b, ~a, a<<1, a>>1)
big = a if a > b else b                # ternary
print("Ternary:", big)
l = [1, 2, 3]
print("Membership:", 2 in l, 5 not in l)
p = [1, 2, 3]; q = l; r = [1, 2, 3]
print("Identity:", l is q, l is r, l is not r)
# Output
# Arithmetic: 13 7 30 3.33.. 3 1 1000
# Bitwise: 2 11 9 -11 20 5
```

Exp 5: Add and multiply complex numbers

```
c1 = complex(2, 3)              # 2+3j
c2 = complex(4, -1)             # 4-1j
print("Sum     =", c1 + c2)     # (6+2j)
print("Product =", c1 * c2)     # (11+10j)
# (2+3j)(4-j) = 8 -2j +12j -3j^2 = 11 + 10j
print("Real part:", c1.real, " Imag:", c1.imag)
```

Exp 6: Multiplication table of a number

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
# Output (n = 7):  7 x 1 = 7 ... 7 x 10 = 70
```

Revision: Python 3.x - created by Guido van Rossum (1991). input() returns str. 'is' = identity, '==' = equality. Strongly + dynamically typed. Indentation = blocks. try-except-else-finally for exceptions.

UNIT - II : FUNCTIONS, STRINGS AND LISTS

I. Functions

* A FUNCTION is a named, reusable block of code that performs a specific task. Advantages: code reusability, modularity, easy debugging and maintenance, less repetition. Types: built-in functions, functions in modules, user defined functions.

Built-in functions (commonly used)

Function	Purpose	Example
print(), input()	Output / input	print('hi')
len(x)	Number of items	len('abc') -> 3
type(x), id(x)	Type / address	type(5) -> int
int(), float(), str()	Type conversion	int('7') -> 7
abs(x), pow(x,y)	Absolute value, power	abs(-4) -> 4
max(), min(), sum()	Largest, smallest, total	sum([1,2,3]) -> 6
round(x, n)	Round to n digits	round(3.14159, 2) -> 3.14
sorted(), reversed()	Sort / reverse	sorted([3,1,2])
range(), enumerate(), zip()	Iteration helpers	range(5)
chr(), ord()	Char <-> ASCII	ord('A') -> 65
divmod(a, b)	Quotient and remainder	divmod(7,2) -> (3,1)
eval(), exec()	Evaluate string code	eval('2+3') -> 5

Commonly used modules

* MODULE = a .py file containing functions, classes, variables that can be imported. Forms: import math; import math as m; from math import sqrt, pi; from math import * (not recommended).

```
import math, random, datetime, os, sys
print(math.sqrt(16), math.pi, math.ceil(4.2), math.floor(4.8))
print(math.factorial(5), math.gcd(12, 18), math.pow(2, 3))
print(random.randint(1, 10), random.random())
print(random.choice(['a', 'b', 'c']))
print(datetime.datetime.now())
```

* Other modules: statistics (mean, median), time (sleep, time), calendar, sys (argv), os, json, re, collections, itertools, numpy, pandas. dir(math) lists contents, help(math.sqrt) shows help.

Function definition and calling

* Syntax: def function_name(parameters): body ... return value. PARAMETERS appear in the definition (formal); ARGUMENTS are the values passed when calling (actual). A function runs only when CALLED. Docstring (first string) documents the function.

```
def add(a, b):
    '''returns sum of two numbers'''
    return a + b

result = add(5, 3)        # function call
print(result)             # 8
print(add.__doc__)
```

return statement and void function

* return sends a value back to the caller and ends the function. A function with NO return (or plain return) is a VOID function - it returns None automatically. A function can return multiple values as a tuple.

```
def greet(name):          # void function
    print("Hello", name)

x = greet("Ravi")         # prints Hello Ravi
print(x)                  # None

def square(n):
    return n * n          # fruitful function
```

Scope and lifetime of variables

* LOCAL variable: created inside a function, exists only while the function runs. GLOBAL variable: defined outside, visible everywhere; to modify a global inside a function use the 'global' keyword. nonlocal is for nested functions. Name lookup follows LEGB rule: Local -> Enclosing -> Global -> Built-in.

```
count = 10                # global

def show():
    count = 5             # local, different variable
    print("inside:", count)    # 5

def change():
    global count
    count = 99            # modifies global

show();  print("outside:", count)   # 10
change(); print(count)              # 99
```

Default parameters

* A parameter can have a default value used when the argument is not supplied. Default parameters must come AFTER non-default ones.

```
def power(base, exp=2):
    return base ** exp

print(power(5))         # 25  (exp = 2)
print(power(2, 5))      # 32
```

Keyword arguments

* Arguments passed as name=value, so the order does not matter. Positional arguments must come before keyword arguments.

```
def student(name, age, city="Hyd"):
    print(name, age, city)

student(age=20, name="Anu")            # Anu 20 Hyd
student("Raj", city="Pune", age=21)    # Raj 21 Pune
```

*args and **kwargs

* *args collects any number of POSITIONAL arguments into a TUPLE. **kwargs collects any number of KEYWORD arguments into a DICTIONARY. Order in definition: normal, *args, default, **kwargs.

```
def total(*args):
    s = 0
    for a in args:
        s += a
    return s
print(total(1, 2, 3, 4))       # 10

def info(**kwargs):
    for k, v in kwargs.items():
        print(k, "=", v)
info(name="Anu", age=20)       # name = Anu / age = 20
```

Command line arguments

* Values passed to the program when it is run from the terminal. In the sys module, sys.argv is a LIST of strings: sys.argv[0] = script name, sys.argv[1:] = arguments. (argparse module is the advanced alternative.)

```
# file: add.py      run:  python add.py 10 20
import sys
print("Script:", sys.argv[0])
print("Count:", len(sys.argv))
total = int(sys.argv[1]) + int(sys.argv[2])
print("Sum =", total)            # Sum = 30
```

Recursion and lambda functions

* RECURSION: a function that calls itself; must have a BASE CASE to stop. Default recursion limit is about 1000 (sys.getrecursionlimit()). LAMBDA: small anonymous one-line function: lambda arguments: expression; often used with map(), filter(), sorted().

```
def fact(n):
    if n <= 1:              # base case
        return 1
    return n * fact(n - 1)
print(fact(5))              # 120

sq = lambda x: x * x
print(sq(6))                                    # 36
print(list(map(lambda x: x*2, [1, 2, 3])))        # [2, 4, 6]
print(list(filter(lambda x: x%2==0, range(10))))  # [0, 2, 4, 6, 8]
```

Pass by	Behaviour in Python
Immutable arg (int, str, tuple)	Changes inside function do NOT affect the caller
Mutable arg (list, dict, set)	Changes inside function DO affect the caller (reference is passed)

```
def modify(lst):
    lst.append(99)
nums = [1, 2]
modify(nums)
print(nums)                 # [1, 2, 99]
```

2. Strings

Creating and storing strings

* A string is an immutable sequence of characters (Unicode) in single, double or triple quotes. Stored in a variable; triple quotes allow

multi-line. Escape sequences: \n newline, \t tab, \\ backslash, \' \". Raw string r'C:\new' ignores escapes.

```
s1 = 'Python'
s2 = "Programming"
s3 = '''Multi
line'''
print(len(s1), type(s1))      # 6 <class 'str'>
```

Basic string operations

```
a, b = "Hello", "World"
print(a + " " + b)       # Hello World   (concatenation)
print(a * 3)             # HelloHelloHello  (repetition)
print('e' in a)          # True          (membership)
print(a == "Hello", a < b)   # True True (comparison)
for ch in a: print(ch, end="-")   # H-e-l-l-o-
```

Accessing characters by index

* Index starts at 0 from the left; negative index starts at -1 from the right. Strings are IMMUTABLE: s[0] = 'J' gives TypeError.

```
s = "PYTHON"
#    P  Y  T  H  O  N
#    0  1  2  3  4  5
#   -6 -5 -4 -3 -2 -1
print(s[0], s[3], s[-1])      # P H N
```

String slicing and joining

* Slice s[start:stop:step] - start included, stop EXCLUDED, default step 1; negative step reverses. join() joins elements of an iterable using the string as separator.

```
s = "PYTHON"
print(s[1:4])      # YTH
print(s[:3])       # PYT
print(s[2:])       # THON
print(s[::2])      # PTO
print(s[::-1])     # NOHTYP   (reverse)
print("-".join(["a", "b", "c"]))   # a-b-c
print(" ".join(["I", "love", "Python"]))
```

String methods

Method	Meaning	Example -> result
upper(), lower()	Change case	'abc'.upper() -> 'ABC'
title(), capitalize()	Title / first capital	'hi all'.title() -> 'Hi All'
swapcase()	Swap case	'aB'.swapcase() -> 'Ab'
strip(), lstrip(), rstrip()	Remove spaces	' hi '.strip() -> 'hi'
find(s), index(s)	Position (-1 / error)	'hello'.find('l') -> 2
count(s)	Occurrences	'banana'.count('a') -> 3
replace(old, new)	Replace	'aab'.replace('a','x') -> 'xxb'
split(sep)	String -> list	'a b c'.split() -> ['a','b','c']
startswith(), endswith()	Test prefix / suffix	'file.py'.endswith('.py') -> True
isalpha(), isdigit(), isalnum()	Character tests	'123'.isdigit() -> True
islower(), isupper(), isspace()	Case / space tests	'ab'.islower() -> True
center(w), ljust(w), rjust(w)	Alignment	'hi'.center(6,'*') -> '**hi**'
zfill(w)	Pad with zeros	'7'.zfill(3) -> '007'

```
s = "  Python is fun  "
print(s.strip().upper())          # PYTHON IS FUN
print(s.split())                  # ['Python', 'is', 'fun']
print("banana".count("an"))       # 2
print("hello".replace("l", "L"))  # heLLo
print("madam" == "madam"[::-1])   # True  (palindrome check)
print("a,b,c".split(","))         # ['a', 'b', 'c']
print("Hello".center(11, "*"))    # ***Hello***
```

Formatting strings

```
name, mark = "Anu", 87.456
print("Name: %s, Marks: %.1f" % (name, mark))     # % operator
print("Name: {}, Marks: {:.2f}".format(name, mark))   # format()
print("{1} scored {0}".format(mark, name))        # indexes
print(f"{name} scored {mark:.2f}")                # f-string
print(f"{255:b} {255:o} {255:x} {1234567:,}")     # 11111111 377 ff 1,23
print(f"{name:>10}|{name:<10}|{name:^10}|")       # align
```

3. Lists

Creating lists

* A LIST is an ordered, MUTABLE collection of items (can be of different types), written in square brackets; allows duplicates. Created by [], list(), or list comprehension.

```
a = [10, 20, 30]
b = ["Ram", 25, 5.8, True]       # mixed types
c = list("abc")                  # ['a', 'b', 'c']
d = list(range(1, 6))            # [1, 2, 3, 4, 5]
e = [x * x for x in range(5)]    # [0, 1, 4, 9, 16]
nested = [[1, 2], [3, 4]]        # 2-D list; nested[1][0] = 3
```

Basic list operations

```
x = [1, 2, 3]; y = [4, 5]
print(x + y)          # [1, 2, 3, 4, 5]  concatenation
print(x * 2)          # [1, 2, 3, 1, 2, 3] repetition
print(2 in x)         # True
print(len(x))         # 3
for i in x: print(i, end=" ")
x[0] = 100            # lists are mutable
```

Indexing and slicing in lists

```
L = [10, 20, 30, 40, 50]
print(L[0], L[-1])       # 10 50
print(L[1:4])            # [20, 30, 40]
print(L[:2], L[3:])      # [10, 20] [40, 50]
print(L[::-1])           # [50, 40, 30, 20, 10]
L[1:3] = [21, 31]        # slice assignment
```

Built-in functions used on lists

```
L = [4, 1, 9, 3]
print(len(L), max(L), min(L), sum(L))   # 4 9 1 17
print(sorted(L))                        # [1, 3, 4, 9]  (new list)
print(list(reversed(L)))                # [3, 9, 1, 4]
print(any(L), all(L))                   # True True
for i, v in enumerate(L): print(i, v)
```

List methods

Method	Action
append(x)	Add x at the end
insert(i, x)	Insert x at index i
extend(list)	Add all items of another list
remove(x)	Delete first occurrence of x
pop(i)	Remove and return item at i (default last)
index(x)	Index of first x
count(x)	Number of x
sort(reverse=False)	Sort in place
reverse()	Reverse in place
copy(), clear()	Shallow copy / empty the list

```
L = [3, 1, 2]
L.append(5); L.insert(1, 9); L.extend([7, 8])
print(L)            # [3, 9, 1, 2, 5, 7, 8]
L.remove(9); print(L.pop())      # 8
L.sort(); print(L)              # [1, 2, 3, 5, 7]
L.reverse(); print(L)           # [7, 5, 3, 2, 1]
```

del statement

```
L = [10, 20, 30, 40, 50]
del L[1]          # [10, 30, 40, 50]
del L[1:3]        # [10, 50]
del L             # deletes the whole list (NameError afterwards)
```

List comprehension, copying and 2-D lists

```
sq  = [x**2 for x in range(1, 6)]              # [1, 4, 9, 16, 25]
ev  = [x for x in range(20) if x % 2 == 0]     # even numbers
a = [1, 2, 3]
b = a             # same list (reference): change in b changes a
c = a.copy()      # new list (shallow copy), also a[:] or list(a)
b.append(4); print(a, c)      # [1, 2, 3, 4] [1, 2, 3]
m = [[1, 2, 3], [4, 5, 6]]
print(m[1][2], len(m), len(m[0]))     # 6 2 3
```

List	String
Mutable - can be changed in place	Immutable - new string created on change
Square brackets [ ]	Quotes ' ' or " "
Items of any type	Characters only

pop() / remove() / del - all delete; pop returns the value, del uses index or slice, remove uses the value. sort() changes the list, sorted() returns a new one.

4. Sample Experiments (7 - 12)

Exp 7: Function with multiple return values

```
def calc(a, b):
    return a + b, a - b, a * b, a / b      # returns a tuple

s, d, p, q = calc(20, 4)
print("Sum:", s, "Diff:", d, "Prod:", p, "Quot:", q)
# Output: Sum: 24 Diff: 16 Prod: 80 Quot: 5.0
```

Exp 8: Function using default arguments

```
def interest(p, r=5, t=1):
    return p * r * t / 100

print(interest(1000))              # 50.0
print(interest(1000, 8))           # 80.0
print(interest(1000, 8, 3))        # 240.0
print(interest(1000, t=2))         # 100.0
```

Exp 9: Length of string without library function

```
s = input("Enter a string: ")
count = 0
for ch in s:
    count += 1
print("Length of the string =", count)
# Input: python -> Length of the string = 6
```

Exp 10: Check if a substring is present

```
s = input("Enter string: ")
sub = input("Enter substring: ")
if sub in s:
    print("Substring found")
else:
    print("Substring not found")
# without 'in': loop  for i in range(len(s)-len(sub)+1):
#                     if s[i:i+len(sub)] == sub: found
```

Exp 11: List operations - addition, insertion, slicing

```
L = [10, 20, 30, 40, 50]
L.append(60)              # addition at end
L.insert(2, 25)           # insertion at index 2
print("After add / insert:", L)
print("L[1:4]  =", L[1:4])
print("L[::2]  =", L[::2])
print("L[-3:]  =", L[-3:])
# [10, 20, 25, 30, 40, 50, 60] -> [20, 25, 30] ...
```

Exp 12: Five built-in functions on a list

```
L = [45, 12, 78, 33, 5]
print("len    :", len(L))          # 5
print("max    :", max(L))          # 78
print("min    :", min(L))          # 5
print("sum    :", sum(L))          # 173
print("sorted :", sorted(L))       # [5, 12, 33, 45, 78]
```

Important questions

* 1. Explain default, keyword, *args and **kwargs arguments with examples.

* 2. Scope and lifetime of variables (local, global, LEGB).
* 3. List any 8 string methods with examples; explain string slicing.
* 4. Differentiate list and string; explain list methods with examples.
* 5. What are command line arguments? How are they used in Python?

Revision: def / return / None; LEGB scope; *args (tuple), **kwargs (dict); sys.argv; strings immutable, slicing [start:stop:step]; lists mutable - append, insert, extend, remove, pop, sort, del.

UNIT - III : DICTIONARIES, TUPLES AND SETS

I. Dictionaries

* A DICTIONARY (dict) is an unordered-by-index (insertion ordered from Python 3.7), MUTABLE collection of KEY : VALUE pairs inside curly braces { }. Keys must be UNIQUE and IMMUTABLE (int, str, tuple); values can be any type and may repeat. Access is by key (hash table - very fast lookup).

Creating a dictionary

```
d1 = {}                                    # empty dict
d2 = {"name": "Ravi", "age": 20, "city": "Hyd"}
d3 = dict(a=1, b=2)                        # using dict()
d4 = dict([("x", 10), ("y", 20)])          # from list of tuples
d5 = dict.fromkeys(["a", "b", "c"], 0)     # {'a':0,'b':0,'c':0}
sq = {x: x*x for x in range(1, 5)}         # comprehension
print(sq)                                  # {1:1, 2:4, 3:9, 4:16}
```

Accessing and modifying key:value pairs

```
d = {"name": "Ravi", "age": 20}
print(d["name"])              # Ravi   (KeyError if missing)
print(d.get("city"))          # None   (no error)
print(d.get("city", "NA"))    # NA     (default value)
d["age"] = 21                 # modify existing
d["city"] = "Hyd"             # add new pair
d.update({"age": 22, "phone": 99})   # update / add many
print(d)
```

```
for k in d:                     # keys
    print(k, end=" ")
for k, v in d.items():          # key and value
    print(k, "->", v)
print("age" in d)               # True  (membership tests keys)
```

Built-in functions used on dictionaries

Function	Result for d = {'a':3,'b':1,'c':2}
len(d)	3 (number of pairs)
max(d), min(d)	'c', 'a' (compares KEYS)
sorted(d)	['a', 'b', 'c'] (sorted keys)
sum(d.values())	6
list(d), tuple(d)	['a','b','c'], ('a','b','c')
any(d), all(d)	True, True (on keys)
str(d), type(d)	string form, <class 'dict'>

Dictionary methods

Method	Action
keys()	View of all keys
values()	View of all values
items()	View of (key, value) tuples
get(k, default)	Value of key k, or default / None
update(d2)	Add / change pairs from d2
pop(k)	Remove key k and return its value
popitem()	Remove and return last inserted pair
setdefault(k, v)	Return value if k exists, else insert k:v
copy()	Shallow copy
clear()	Remove all pairs
fromkeys(seq, v)	New dict with keys from seq

```
d = {"a": 1, "b": 2, "c": 3}
print(d.keys())          # dict_keys(['a', 'b', 'c'])
print(d.values())        # dict_values([1, 2, 3])
print(d.items())         # dict_items([('a',1),('b',2),('c',3)])
print(d.pop("b"))        # 2
print(d.popitem())       # ('c', 3)
d.setdefault("z", 26)
print(d)                 # {'a': 1, 'z': 26}
```

del statement

```
d = {"a": 1, "b": 2, "c": 3}
del d["b"]           # removes the key 'b'  -> {'a':1,'c':3}
d.clear()            # {} - empties the dictionary
del d                # deletes the dictionary object itself
```

Nested dictionary

```
students = {
    101: {"name": "Anu", "marks": 85},
    102: {"name": "Raj", "marks": 72},
}
print(students[101]["name"])      # Anu
students[102]["marks"] = 80
for roll, info in students.items():
    print(roll, info["name"], info["marks"])
```

Common dictionary programs

```
# 1. word / character frequency
text = "banana"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)                 # {'b': 1, 'a': 3, 'n': 2}

# 2. sort by key and by value
d = {"x": 30, "a": 10, "m": 20}
print(sorted(d.items()))                          # by key
print(sorted(d.items(), key=lambda p: p[1]))      # by value
print(dict(sorted(d.items(), key=lambda p: p[1], reverse=True)))

# 3. merge two dictionaries
d1 = {"a": 1}; d2 = {"b": 2, "a": 9}
print({**d1, **d2})         # {'a': 9, 'b': 2}   (d1 | d2 in 3.9+)

# 4. invert keys and values
inv = {v: k for k, v in {"a": 1, "b": 2}.items()}   # {1:'a', 2:'b'}
```

Operation	Time	Note
d[key], get()	O(1) average	hash lookup
d[key] = v	O(1) average	insert / update
key in d	O(1) average	fast membership
key in list	O(n)	slow - scans the list

Keys must be immutable & unique (list as key -> TypeError: unhashable). Duplicate key in the literal keeps the LAST value.

2. Tuples

* A TUPLE is an ordered, IMMUTABLE sequence of items in round brackets ( ). Once created, items cannot be added, changed or removed. Allows duplicates and mixed types. Faster and uses less memory than a list; can be used as dictionary key; used for fixed data (days of week, coordinates).

Creating tuples

```
t1 = ()                         # empty tuple
t2 = (10, 20, 30)
t3 = 5, "abc", 2.5              # packing (brackets optional)
t4 = (7,)                       # single item needs a comma!
t5 = (7)                        # NOT a tuple - just int 7
a, b, c = t2                    # unpacking
print(type(t4), type(t5))       # tuple, int
```

Basic tuple operations

```
x = (1, 2, 3); y = (4, 5)
print(x + y)         # (1, 2, 3, 4, 5)  concatenation
print(x * 2)         # (1, 2, 3, 1, 2, 3)
print(2 in x)        # True
print(len(x))        # 3
for i in x: print(i, end=" ")
x[0] = 9             # TypeError: immutable
```

tuple() function, indexing and slicing

```
print(tuple([1, 2, 3]))       # (1, 2, 3)  list -> tuple
print(tuple("hi"))            # ('h', 'i')
print(tuple(range(4)))        # (0, 1, 2, 3)
t = (10, 20, 30, 40, 50)
print(t[0], t[-1])            # 10 50
print(t[1:4])                 # (20, 30, 40)
print(t[::-1])                # (50, 40, 30, 20, 10)
```

Built-in functions and methods on tuples

```
t = (4, 1, 9, 1)
print(len(t), max(t), min(t), sum(t))    # 4 9 1 15
print(sorted(t))              # [1, 1, 4, 9]   (returns LIST)
print(t.count(1))             # 2
print(t.index(9))             # 2
```

* Only 2 tuple methods: count() and index().

```
# nested tuple, swap and returning many values
t = ((1, 2), (3, 4), "end")
print(t[1][0])                  # 3
a, b = 5, 9
a, b = b, a                     # swap using tuple packing
def stats(nums):
    return min(nums), max(nums), sum(nums)
lo, hi, tot = stats([4, 8, 1])    # tuple unpacking
first, *rest = (1, 2, 3, 4)       # first=1, rest=[2,3,4]
```

* Tuples can contain mutable objects: t = (1, [2, 3]); t[1].append(4) works -> (1, [2, 3, 4]); but t[1] = [9] fails. del t deletes the whole tuple (cannot delete single items).

Relation between tuples and lists

Tuple	List
Immutable	Mutable
( ) round brackets	[ ] square brackets
Faster, less memory	Slower, more memory
Only count(), index() methods	Many methods (append, sort ...)
Can be a dictionary key	Cannot be a dictionary key
Fixed / heterogeneous data	Variable / homogeneous data

```
L = list((1, 2, 3))             # tuple -> list (to modify)
L.append(4)
T = tuple(L)                    # list -> tuple
print(L, T)                     # [1,2,3,4] (1,2,3,4)
```

Relation between tuples and dictionaries

* dict.items() returns (key, value) TUPLES; a list of 2-item tuples can be converted to a dict with dict(); tuples can be used as KEYS of a dictionary (because immutable).

```
d = {"a": 1, "b": 2}
lst = list(d.items())              # [('a', 1), ('b', 2)]
d2 = dict(lst)                     # back to dict
loc = {(17.38, 78.48): "Hyderabad", (28.6, 77.2): "Delhi"}
print(loc[(28.6, 77.2)])           # Delhi
```

zip() function

* zip() combines items of two or more iterables position by position into TUPLES; stops at the shortest. Result is an iterator - wrap in list(), dict(). zip(*pairs) unzips.

```
names = ["Anu", "Raj", "Sam"]
marks = [85, 72, 90]
z = list(zip(names, marks))
print(z)              # [('Anu',85), ('Raj',72), ('Sam',90)]
print(dict(zip(names, marks)))    # {'Anu':85,'Raj':72,'Sam':90}
n2, m2 = zip(*z)      # unzip
for n, m in zip(names, marks):
    print(n, m)
```

3. Sets

* A SET is an UNORDERED collection of UNIQUE items in curly braces { }. Mutable (add / remove) but elements must be immutable; no

indexing or slicing; duplicates removed automatically. Used for membership tests and mathematical set operations.

```
s1 = {1, 2, 3, 3, 2}          # duplicates removed
print(s1)                     # {1, 2, 3}
s2 = set([4, 5, 5, 6])        # {4, 5, 6}
s3 = set("hello")             # {'h','e','l','o'}
empty = set()                 # NOT {} (that is an empty dict)
print(len(s1), 2 in s1)       # 3 True
print(list(set([1, 1, 2, 3, 3])))   # remove duplicates
```

Set methods

Method	Action
add(x)	Add one element
update(iter)	Add many elements
remove(x)	Delete x (KeyError if absent)
discard(x)	Delete x (no error if absent)
pop()	Remove and return an arbitrary element
clear(), copy()	Empty / shallow copy
union(s2) |	All elements of both
intersection(s2) &	Common elements
difference(s2) -	In s1 but not in s2
symmetric_difference(s2) ^	In either, not both
issubset(), issuperset(), isdisjoint()	Relationship tests (True / False)

```
A = {1, 2, 3, 4}; B = {3, 4, 5, 6}
print(A | B)        # {1, 2, 3, 4, 5, 6}   union
print(A & B)        # {3, 4}               intersection
print(A - B)        # {1, 2}               difference
print(A ^ B)        # {1, 2, 5, 6}         symmetric difference
print({3, 4} <= A)  # True                 subset
A.add(10); A.discard(1); A.remove(2)
print(A)            # {3, 4, 10}
```

```
# practical uses of sets
nums = [1, 2, 2, 3, 4, 4, 5]
print(list(set(nums)))                  # remove duplicates
a = set("python"); b = set("typhoon")
print(a & b)                            # common letters
print(a - b)                            # letters only in a
print(len(set("mississippi")))          # 4 unique letters
print(set(nums) == {1, 2, 3, 4, 5})     # True (order ignored)
s = {1, 2}
s.update([3, 4], {5})                   # add many items
print(sorted(s, reverse=True))          # [5, 4, 3, 2, 1]
```

Set operation	Operator	Method	Venn meaning
Union	A | B	A.union(B)	everything in A or B
Intersection	A & B	A.intersection(B)	only the overlap
Difference	A - B	A.difference(B)	A without the overlap
Symmetric diff.	A ^ B	A.symmetric_difference(B)	both except overlap

Frozenset

* A FROZENSET is an IMMUTABLE set: created with frozenset(iterable); supports all non-modifying set operations (union, intersection, difference, issubset) but NOT add / remove. Because it is hashable, a frozenset can be a dictionary key or an element of another set.

```
f = frozenset([1, 2, 3, 2])
print(f)                          # frozenset({1, 2, 3})
print(f | frozenset([5]))         # frozenset({1, 2, 3, 5})
f.add(4)                          # AttributeError: no attribute 'add'
d = {frozenset([1, 2]): "pair"}   # allowed as a key
```

	List	Tuple	Set	Dictionary
Brackets	[ ]	( )	{ }	{k: v}
Ordered	Yes	Yes	No	Insertion order
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	Yes	Yes	No	By key

4. Sample Experiments (13 - 17)

Exp 13: Create tuples (name, age, address, college) and concatenate

```
m1 = ("Anu", 20, "Hyderabad", "JNTU")
m2 = ("Raj", 21, "Warangal", "NIT")
result = m1 + m2
print("Member 1:", m1)
print("Member 2:", m2)
print("Concatenated tuple:", result)
# ('Anu', 20, 'Hyderabad', 'JNTU', 'Raj', 21, 'Warangal', 'NIT')
```

Exp 14: Count the vowels in a string (no control flow)

* No if / for / while allowed - use string method count() and sum(), or len() with a set / map.

```
s = input("Enter a string: ").lower()
vowels = s.count("a") + s.count("e") + s.count("i") + \
         s.count("o") + s.count("u")
print("Number of vowels:", vowels)
# Input: Python Programming -> Number of vowels: 4
# another way: print(sum(map(s.count, "aeiou")))
```

Exp 15: Check if a given key exists in a dictionary

```
d = {"name": "Ravi", "age": 20, "city": "Hyd"}
key = input("Enter key to search: ")
if key in d:
    print("Key exists, value =", d[key])
else:
    print("Key does not exist")
# Input: age -> Key exists, value = 20
```

Exp 16: Add a new key-value pair to an existing dictionary

```
d = {"a": 1, "b": 2}
print("Before:", d)
k = input("New key: ")
v = int(input("New value: "))
d[k] = v                          # or d.update({k: v})
print("After :", d)
# Input: c 3 -> After : {'a': 1, 'b': 2, 'c': 3}
```

Exp 17: Sum all the items in a dictionary

```
d = {"a": 100, "b": 200, "c": 300}
total = sum(d.values())
print("Sum of all items =", total)       # 600
# using loop
t = 0
for v in d.values():
    t += v
print(t)
```

Output prediction practice

```
print(type({}), type(()), type({1}), type((1,)), type((1)))
# <class 'dict'> <class 'tuple'> <class 'set'> <class 'tuple'> <class
d = {1: "a", 2: "b", 1: "c"}
print(d, len(d))                  # {1: 'c', 2: 'b'} 2
print({1, 2, 3} == {3, 2, 1})     # True
print(list(zip("ab", [1, 2, 3])))    # [('a', 1), ('b', 2)]
print(max({"x": 5, "y": 9}, key=lambda k: {"x": 5, "y": 9}[k]))  # y
```

Important questions

* 1. Explain dictionary creation, accessing and methods with examples.
* 2. Differentiate list, tuple, set and dictionary.
* 3. What is zip()? Explain with an example. What is a frozenset?
* 4. Explain set operations - union, intersection, difference, symmetric difference.

Revision: dict = key:value, keys unique and immutable. Tuple = immutable, single item needs comma. Set = unique, unordered, set() for empty. Frozenset = immutable set. zip() pairs items.

UNIT - IV : FILES AND OBJECT-ORIENTED PROGRAMMING

I. Files

* A FILE is a named location on secondary storage (disk) used to store data permanently. Programs use files to save and read data that must survive after the program ends.

Types of files

* TEXT files: store characters (ASCII / Unicode) as lines ending with newline '\n'; human readable - .txt, .py, .csv, .html. BINARY files: store data as bytes (0 / 1), not readable in an editor - .jpg, .mp3, .exe, .pdf, .dat, .pkl.

Opening and closing a file

* open(filename, mode) returns a file object. Always close with close() or use the WITH statement (closes automatically, even on error - best practice).

Mode	Meaning
'r'	Read (default); file must exist, else FileNotFoundError
'w'	Write; creates new file or ERASES existing content
'a'	Append; adds at the end, creates if not present
'x'	Exclusive create; error if the file exists
'r+' 'w+' 'a+'	Read and write (both)
't' / 'b'	Text (default) / Binary - 'rb', 'wb', 'ab'

Creating and reading text data

```
f = open("demo.txt", "w")          # create / write
f.write("Hello Python\n")
f.write("File handling\n")
f.close()

with open("demo.txt", "r") as f:   # read whole file
    data = f.read()
print(data)
```

File methods to read and write data

Method	Action
read() / read(n)	Whole file / first n characters as a string
readline()	Next one line (with \n)
readlines()	All lines as a LIST of strings
write(str)	Write a string (returns number of chars)
writelines(list)	Write a list of strings (no automatic \n)
tell()	Current position of file pointer
seek(offset, whence)	Move the file pointer; seek(0) = start
flush(), close()	Force write buffer / close file
name, mode, closed	File attributes

```
with open("demo.txt") as f:
    print(f.readline(), end="")        # first line
    print(f.tell())                    # pointer position
    rest = f.readlines()               # remaining lines (list)
f = open("demo.txt")
for line in f:                         # memory efficient loop
    print(line.strip())
f.close()
with open("demo.txt", "a") as f:       # append
    f.write("New line added\n")
```

Reading and writing binary files

```
data = bytes([65, 66, 67, 68])
with open("bin.dat", "wb") as f:
    f.write(data)
with open("bin.dat", "rb") as f:
    print(f.read())                    # b'ABCD'

# copy an image (binary)
with open("a.jpg", "rb") as src, open("b.jpg", "wb") as dst:
    dst.write(src.read())
```

Pickle module

* PICKLE serialises (pickling) a Python object (list, dict, class object) into a byte stream and stores it in a binary file; unpickling restores the object. pickle.dump(obj, file) and pickle.load(file). Open in 'wb' / 'rb'. Never unpickle data from untrusted sources (security risk).

```
import pickle
student = {"name": "Anu", "marks": [85, 90, 78]}
with open("stu.pkl", "wb") as f:
    pickle.dump(student, f)            # pickling
with open("stu.pkl", "rb") as f:
    obj = pickle.load(f)               # unpickling
print(obj["name"], obj["marks"])
```

Reading and writing CSV files

* CSV = Comma Separated Values, a plain text table (each line a row, items separated by commas). Use the csv module: csv.reader / csv.writer (lists) and csv.DictReader / csv.DictWriter (dictionaries). Open with newline='' when writing.

```
import csv
rows = [["Name", "Marks"], ["Anu", 85], ["Raj", 72]]
with open("m.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["ID", "Name"])         # single row
    w.writerows(rows)                  # many rows
with open("m.csv", "r") as f:
    for row in csv.reader(f):
        print(row)                     # ['Name', '85'] ...
with open("m.csv") as f:
    for r in csv.DictReader(f):        # first row = keys
        print(r["Name"], r["Marks"])
```

Python os and os.path modules

Function	Purpose
os.getcwd()	Current working directory
os.listdir(path)	List files and folders
os.mkdir(d), os.makedirs(p)	Create directory / nested directories
os.rmdir(d), os.remove(f)	Delete empty directory / file
os.rename(old, new)	Rename a file or directory
os.chdir(path)	Change directory
os.path.exists(p)	True if path exists
os.path.isfile(p), isdir(p)	Is it a file / directory?
os.path.join(a, b)	Join path parts with correct separator
os.path.getsize(p)	Size in bytes
os.path.splitext(p)	Split name and extension
os.path.basename(p), dirname(p)	File name / folder part

```
import os
print(os.getcwd())
if not os.path.exists("data"):
    os.mkdir("data")
p = os.path.join("data", "a.txt")
open(p, "w").write("hi")
print(os.path.getsize(p), os.path.splitext(p))   # 2 ('data/a','.txt')
print(os.listdir("data"))                         # ['a.txt']
os.remove(p); os.rmdir("data")
```

2. Object-Oriented Programming (OOP)

* OOP organises a program around OBJECTS (data + functions together) instead of functions only. Main concepts: CLASS, OBJECT, ENCAPSULATION, INHERITANCE, POLYMORPHISM, ABSTRACTION. Benefits: reusability, modularity, security (data hiding), easy maintenance, real-world modelling.

Classes and objects

* CLASS = blueprint / template that defines attributes (variables) and methods (functions). OBJECT = an instance of a class, with its own copy of data. Example: class Car -> objects car1, car2.

Creating classes and objects in Python

```
class Student:                          # class definition
    college = "JNTU"                    # class attribute
    def show(self):                     # method
        print("I am a student")

s1 = Student()                          # object creation
s1.show()                               # method call
print(s1.college, type(s1))
```

Constructor method (__init__)

* __init__(self, ...) is a special method CALLED AUTOMATICALLY when an object is created, to initialise data attributes. self refers to the current object (first parameter of every instance method). Python allows only one __init__ (use default arguments for flexibility). __del__ is the destructor.

```
class Student:
    def __init__(self, name, roll):
        self.name = name              # data (instance) attributes
        self.roll = roll
    def display(self):
        print(self.roll, self.name)

s = Student("Anu", 101)
s.display()                           # 101 Anu
```

Classes with multiple objects

```
s1 = Student("Anu", 101)
s2 = Student("Raj", 102)
s3 = Student("Sam", 103)
for obj in (s1, s2, s3):
    obj.display()                     # each has its own data
print(s1 is s2, id(s1) != id(s2))     # False True
```

Class attributes vs data (instance) attributes

Class attribute	Data (instance) attribute
Defined in class body, outside methods	Defined inside __init__ / methods with self.
SHARED by all objects (one copy)	Separate copy for every object
Accessed by ClassName.attr or object.attr	Accessed by object.attr only
Changing via class affects all objects	Changing affects only that object

```
class Employee:
    company = "TCS"                    # class attribute
    count = 0
    def __init__(self, name):
        self.name = name               # instance attribute
        Employee.count += 1
e1 = Employee("A"); e2 = Employee("B")
print(Employee.count, e1.company)      # 2 TCS
Employee.company = "Infosys"
print(e2.company)                      # Infosys (shared)
e1.name = "Z"; print(e2.name)          # B (not shared)
```

Encapsulation

* Wrapping data and the methods that work on it into ONE unit (class) and restricting direct access to the data = DATA HIDING. Access levels by naming: public (name), protected (_name, convention only), private (__name, name mangled to _Class__name). Use getter / setter methods.

```
class Account:
    def __init__(self, bal):
        self.__balance = bal          # private
    def deposit(self, amt):
        if amt > 0:
            self.__balance += amt
    def get_balance(self):            # getter
        return self.__balance
a = Account(1000)
a.deposit(500)
print(a.get_balance())                # 1500
print(a.__balance)                    # AttributeError
print(a._Account__balance)            # 1500 (name mangling)
```

Inheritance

* A new class (CHILD / derived / sub class) acquires the attributes and methods of an existing class (PARENT / base / super class) -> code reuse. Syntax: class Child(Parent). super() calls the parent's method / constructor. Method overriding: child redefines a parent method.
* Types: SINGLE (one parent), MULTIPLE (many parents: class C(A, B)), MULTILEVEL (A -> B -> C), HIERARCHICAL (one parent, many children), HYBRID.

```
class Person:                                   # parent
    def __init__(self, name):
        self.name = name
    def show(self):
        print("Name:", self.name)

class Student(Person):                          # single inheritance
    def __init__(self, name, roll):
        super().__init__(name)                  # call parent constructor
        self.roll = roll
    def show(self):                             # method overriding
        super().show()
        print("Roll:", self.roll)

Student("Anu", 7).show()
print(issubclass(Student, Person), isinstance(Student("a",1), Person))
```

```
class A: pass
class B(A): pass            # multilevel: A -> B -> C
class C(B): pass
class D: pass
class E(A, D): pass         # multiple inheritance
print(E.__mro__)            # method resolution order
```

Polymorphism

* 'Many forms': the same function / method name behaves differently for different objects or inputs. Types: (1) operator overloading (+ on int adds, on str joins; define __add__), (2) METHOD OVERRIDING

(run-time, via inheritance), (3) duck typing (any object with the needed method works). Python has no compile-time method overloading - use default / *args.

```
class Dog:
    def sound(self): print("Bow Bow")
class Cat:
    def sound(self): print("Meow")
for animal in (Dog(), Cat()):     # same call, different result
    animal.sound()

class P:
    def __init__(self, x): self.x = x
    def __add__(self, o):         # operator overloading
        return P(self.x + o.x)
print((P(2) + P(3)).x)            # 5
print(len("abc"), len([1, 2]))    # one function, many types
```

Special (magic) methods and abstraction

Method	Called when
__init__(self, ...)	object is created
__str__(self)	print(obj) / str(obj)
__repr__(self)	repr(obj), debugging display
__len__(self)	len(obj)
__add__, __sub__, __eq__, __lt__	obj + obj, -, ==, <
__del__(self)	object is destroyed

```
from abc import ABC, abstractmethod
class Vehicle(ABC):                    # abstract class
    @abstractmethod
    def start(self): pass              # must be overridden
class Car(Vehicle):
    def start(self): print("Car starts")
    def __str__(self): return "I am a car"
# v = Vehicle()  -> TypeError (cannot instantiate abstract class)
c = Car(); c.start(); print(c)         # Car starts / I am a car
```

* ABSTRACTION = showing only essential features and hiding implementation details (abstract classes, interfaces). Static method (@staticmethod) needs no self; class method (@classmethod) gets cls - used for alternate constructors.

Important questions

* 1. Explain file modes and file methods; difference between text and binary files.
* 2. What is pickling? Write a program to store and retrieve an object.
* 3. Explain constructor, self, class attribute vs data attribute with examples.
* 4. Explain types of inheritance with programs; what is method overriding?
* 5. Explain encapsulation and polymorphism in Python.

3. Sample Experiments (18 - 23)

Exp 18: Sort words of a file into another file (lower-case)

```
with open("source.txt") as f:
    words = f.read().lower().split()    # lower-case, split
words.sort()
with open("output.txt", "w") as g:
    for w in words:
        g.write(w + "\n")
print("Sorted words written to output.txt")
# source: "Banana apple Cherry" -> output: apple banana cherry
```

Exp 19: Print each line of a file in reverse order

```
with open("source.txt") as f:
    for line in f:
        print(line.rstrip("\n")[::-1])     # reverse characters
# to print lines in reverse order instead:
#   for line in reversed(f.readlines()): print(line)
```

Exp 20: Count characters, words and lines in a file

```
chars = words = lines = 0
with open("source.txt") as f:
    for line in f:
        lines += 1
        words += len(line.split())
        chars += len(line)
print("Lines:", lines, "Words:", words, "Characters:", chars)
```

Exp 21: Array - create, display, append, insert, reverse

```
from array import array
a = array("i", [10, 20, 30, 40])      # create (typecode i = int)
print("Array:", a)
a.append(50)                          # append
a.insert(2, 25)                       # insert at index 2
print("After append/insert:", a.tolist())
a.reverse()                           # reverse
print("Reversed:", a.tolist())        # [50, 40, 30, 25, 20, 10]
for x in a: print(x, end=" ")         # display
```

Exp 22: Add, transpose and multiply two matrices

```
A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
B = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]
add = [[A[i][j] + B[i][j] for j in range(3)] for i in range(3)]
trans = [[A[j][i] for j in range(3)] for i in range(3)]
mul = [[0]*3 for _ in range(3)]
for i in range(3):
    for j in range(3):
        for k in range(3):
            mul[i][j] += A[i][k] * B[k][j]
print("Sum:", add)       # [[10,10,10],[10,10,10],[10,10,10]]
print("Transpose:", trans)   # [[1,4,7],[2,5,8],[3,6,9]]
print("Product:", mul)   # [[30,24,18],[84,69,54],[138,114,90]]
```

Exp 23: Shape class with area and perimeter (Circle, Triangle, Square,

```
import math
class Shape:
    def area(self): pass
    def perimeter(self): pass
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2
    def perimeter(self): return 2 * math.pi * self.r
class Triangle(Shape):
    def __init__(self, a, b, c): self.a, self.b, self.c = a, b, c
    def perimeter(self): return self.a + self.b + self.c
    def area(self):                          # Heron's formula
        s = self.perimeter() / 2
        return math.sqrt(s*(s-self.a)*(s-self.b)*(s-self.c))
class Square(Shape):
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2
    def perimeter(self): return 4 * self.s
for sh in (Circle(5), Triangle(3, 4, 5), Square(4)):
    print(type(sh).__name__, round(sh.area(), 2), round(sh.perimeter()
# Circle 78.54 31.42 / Triangle 6.0 12 / Square 16 16
```

Revision: modes r w a x (+ b); with-statement; pickle.dump / load; csv.reader / writer; os.path.exists. OOP: class, object, __init__, self, encapsulation (__private), inheritance (super()), polymorphism (overriding).

UNIT - V : INTRODUCTION TO DATA SCIENCE

1. Introduction to data science

* DATA SCIENCE = extracting knowledge and insights from structured and unstructured data using statistics, programming and domain knowledge. Life cycle: collect data -> clean (pre-process) -> explore / analyse -> model -> visualise -> communicate / deploy.
* Python is popular for data science because of simple syntax and powerful libraries: NumPy (arrays), Pandas (tables), Matplotlib / Seaborn (plots), SciPy, scikit-learn (machine learning).

2. Functional Programming

* A programming style that builds programs by applying and composing FUNCTIONS, avoiding changing state and mutable data. Features: PURE functions (same input -> same output, no side effects), first class functions (assign to variables, pass as arguments, return from functions), higher order functions, immutability, recursion, lazy iterators.

Higher order functions - map, filter, reduce

Function	Action	Result
map(func, iter)	Apply func to every item	map object (iterator)
filter(func, iter)	Keep items for which func is True	filter object
reduce(func, iter)	Cumulatively combine to a single value (functools)	single value

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
sq   = list(map(lambda x: x * x, nums))          # [1,4,9,16,25]
ev   = list(filter(lambda x: x % 2 == 0, nums))  # [2, 4]
tot  = reduce(lambda a, b: a + b, nums)          # 15
prod = reduce(lambda a, b: a * b, nums)          # 120
print(sq, ev, tot, prod)
print(sorted(["bb", "a", "ccc"], key=len))       # ['a','bb','ccc']
```

* Other tools: lambda, list comprehension, generators (yield), iterators, zip, enumerate, itertools, functools.partial, decorators (functions that wrap other functions), closures.

```
def gen(n):                          # generator - lazy sequence
    for i in range(n):
        yield i * i
print(list(gen(5)))                  # [0, 1, 4, 9, 16]

def make_adder(x):                   # closure / function returning functi
    return lambda y: x + y
add5 = make_adder(5); print(add5(3))    # 8
```

3. JSON and XML in Python

JSON (JavaScript Object Notation)

* Lightweight, text based, language independent data-interchange format; looks like a Python dict: { key: value }. Data types: object, array, string, number, true / false, null. Python's json module:

json.dumps(obj) -> JSON string, json.loads(str) -> Python object, json.dump(obj, file), json.load(file).

JSON	Python
object { }	dict
array [ ]	list
string	str
number (int / real)	int / float
true / false	True / False
null	None

```
import json
data = {"name": "Anu", "age": 20, "skills": ["C", "Python"], "ok": Tru
s = json.dumps(data, indent=2)       # dict -> JSON string
print(s)
obj = json.loads('{"a": 1, "b": [1, 2], "c": null}')   # string -> dic
print(obj["b"], obj["c"])            # [1, 2] None
with open("d.json", "w") as f:
    json.dump(data, f)               # write to file
with open("d.json") as f:
    back = json.load(f)              # read from file
```

XML (eXtensible Markup Language)

* Markup language that stores data in a tree of user-defined TAGS with attributes: <student id='1'><name>Anu</name></student>. Python module: xml.etree.ElementTree - ET.parse(file) / ET.fromstring(str); methods: getroot(), find(), findall(), iter(); element.tag, .attrib, .text.

```
import xml.etree.ElementTree as ET
xml = '''<college>
  <student id="1"><name>Anu</name><marks>85</marks></student>
  <student id="2"><name>Raj</name><marks>72</marks></student>
</college>'''
root = ET.fromstring(xml)
for st in root.findall("student"):
    print(st.attrib["id"], st.find("name").text, st.find("marks").text
new = ET.SubElement(root, "student", id="3")    # add element
ET.SubElement(new, "name").text = "Sam"
print(ET.tostring(root).decode())
```

JSON	XML
Key-value, lighter and faster	Tags, verbose and heavier
Supports arrays, easy to parse	No native arrays; attributes + comments supported
Used in web APIs (REST)	Documents, config, SOAP, RSS

4. NumPy with Python

* NumPy (Numerical Python) = library for fast N-DIMENSIONAL ARRAY (ndarray) processing. Arrays hold items of ONE data type (homogeneous), stored in contiguous memory - much faster and compact than lists; support vectorised operations, broadcasting, linear algebra, random numbers, statistics. Install: pip install numpy; import numpy as np.

Creating arrays

```
import numpy as np
a = np.array([1, 2, 3, 4])              # 1-D from list
b = np.array([[1, 2, 3], [4, 5, 6]])    # 2-D
z = np.zeros((2, 3))                    # all 0.0
o = np.ones((2, 2), dtype=int)          # all 1
r = np.arange(0, 10, 2)                 # [0 2 4 6 8]
l = np.linspace(0, 1, 5)                # [0. .25 .5 .75 1.]
e = np.eye(3)                           # identity 3x3
f = np.full((2, 2), 7)
rn = np.random.rand(2, 2)               # random in [0,1)
```

Attributes: ndim, shape, size, dtype

Attribute	Meaning	For b above
ndim	Number of dimensions (axes)	2
shape	(rows, columns) tuple	(2, 3)
size	Total number of elements	6
dtype	Data type of elements	int64
itemsize	Bytes per element	8

```
print(b.ndim, b.shape, b.size, b.dtype)    # 2 (2, 3) 6 int64
print(b.reshape(3, 2))                     # change shape
print(b.T)                                 # transpose
print(b.flatten())                         # 1-D copy
c = np.array([1, 2, 3], dtype=float)       # [1. 2. 3.]
```

Indexing and slicing

```
a = np.array([10, 20, 30, 40, 50])
print(a[1], a[-1], a[1:4], a[::2])          # basic slicing
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m[1, 2])              # 6        element row 1, col 2
print(m[0])                 # [1 2 3]  first row
print(m[:, 1])              # [2 5 8]  second column
print(m[0:2, 1:3])          # [[2 3] [5 6]]  sub matrix
print(a[[0, 2, 4]])         # integer (fancy) indexing -> [10 30 50]
print(a[a > 25])            # boolean indexing -> [30 40 50]
a[a > 40] = 0               # modify with condition
```

* Slices are VIEWS (share memory); use .copy() for an independent copy.

Operations and aggregate functions

```
x = np.array([1, 2, 3]); y = np.array([4, 5, 6])
print(x + y, x * y, x ** 2, x > 1)       # element-wise
print(np.dot(x, y))                      # 32  (dot product)
a = np.array([[3, 1, 4], [1, 5, 9]])
print(a.min(), a.max(), a.sum())         # 1 9 23
print(a.sum(axis=0), a.sum(axis=1))      # column / row sums
print(a.mean(), np.median(a), a.std())   # statistics
print(np.cumsum([1, 2, 3, 4]))           # [1 3 6 10]  cumulative sum
print(a.argmax(), np.sort(a))            # index of max, sort
```

Broadcasting, random numbers and linear algebra

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a + 10)                          # scalar broadcast to all items
print(a + np.array([100, 200, 300]))    # row vector added to each row
np.random.seed(1)
print(np.random.randint(1, 10, 5))  # 5 random integers
print(np.random.randn(3))           # normal distribution
A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
print(A @ B)                           # matrix product [[19 22] [43 50]]
print(np.linalg.det(A), np.linalg.inv(A))
print(np.sqrt(A), np.exp(A), np.sin(A))     # universal functions
print(np.concatenate([A, B]), np.vstack([A, B]), np.hstack([A, B]))
```

	List	NumPy array
Types	Mixed	Same type
Speed / memory	Slower, more	Faster, compact
[1,2] * 2	[1,2,1,2]	[2 4] (element-wise)
Math on whole data	Needs loops	Vectorised, broadcasting

5. Pandas

* Pandas = library for data manipulation and analysis, built on NumPy. Two main structures: SERIES (1-D labelled array) and DATAFRAME (2-D table with rows and labelled columns, like Excel / SQL table; columns can have different types). import pandas as pd; pip install pandas.

```
import pandas as pd
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s["b"], s.mean())                  # 20 20.0
data = {"Name": ["Anu", "Raj", "Sam", "Ria"],
        "Age": [20, 21, 19, 22],
        "Marks": [85, 72, 90, 66]}
df = pd.DataFrame(data)
print(df)
```

```
   Name  Age  Marks
0   Anu   20     85
1   Raj   21     72
2   Sam   19     90
3   Ria   22     66
```

Reading / writing and exploring

```
df = pd.read_csv("data.csv")       # also read_excel, read_json
df.to_csv("out.csv", index=False)
print(df.head(3))        # first 3 rows (default 5);  df.tail() last r
print(df.shape, df.columns.tolist(), df.dtypes)
df.info()                # columns, non-null counts, types
print(df.describe())     # count mean std min quartiles max
print(df.isnull().sum()) # missing values per column
df = df.dropna()         # or df.fillna(0)
```

Data selection operations

```
print(df["Name"])                    # one column (Series)
print(df[["Name", "Marks"]])         # several columns (DataFrame)
print(df.loc[1])                     # row by LABEL
print(df.loc[0:2, ["Name", "Age"]])   # label slice (end included)
print(df.iloc[1:3, 0:2])             # by POSITION (end excluded)
print(df.iloc[0, 2])                 # 85
print(df[df["Marks"] > 70])          # Boolean filter
print(df[(df.Age > 19) & (df.Marks > 70)])   # multiple conditions
df["Result"] = df["Marks"] >= 70  # new column
df = df.sort_values("Marks", ascending=False)
print(df.groupby("Result")["Marks"].mean())
```

loc	iloc
Label based	Integer position based
Slice includes the END label	Slice excludes the end position
df.loc[row_label, col_label]	df.iloc[row_no, col_no]

Cleaning, grouping, merging

```
df = pd.DataFrame({"dept": ["CSE", "ECE", "CSE", "ECE", "CSE"],
                   "sal": [50, 40, None, 45, 60]})
print(df["sal"].isnull().sum())           # 1 missing
df["sal"] = df["sal"].fillna(df["sal"].mean())    # fill missing
print(df.groupby("dept")["sal"].agg(["mean", "max"]))
print(df["dept"].value_counts())          # CSE 3, ECE 2
df["bonus"] = df["sal"].apply(lambda x: x * 0.1)
df = df.rename(columns={"sal": "salary"}).drop(columns=["bonus"])
other = pd.DataFrame({"dept": ["CSE", "ECE"], "hod": ["Rao", "Devi"]})
print(pd.merge(df, other, on="dept"))     # join like SQL
print(df.drop_duplicates(), df.T.shape)
```

Pandas function	Use
head(n), tail(n)	First / last n rows
info(), describe()	Structure and statistics
loc, iloc	Select by label / position
sort_values(), groupby()	Sort / aggregate
dropna(), fillna()	Handle missing data
merge(), concat()	Combine DataFrames
apply(), map()	Apply a function
value_counts(), unique()	Frequency / distinct values
corr()	Correlation between numeric columns

Plotting with matplotlib

* matplotlib.pyplot draws 2-D graphs: plot() line, scatter() points, bar(), hist(), pie(); add title(), xlabel(), ylabel(), legend(), grid(); show() displays. A SCATTER plot shows how one attribute changes with respect to another (relationship / correlation).

```
import matplotlib.pyplot as plt
plt.scatter(df["Age"], df["Marks"], color="red")
plt.title("Age vs Marks"); plt.xlabel("Age"); plt.ylabel("Marks")
plt.show()
plt.plot(df["Age"], df["Marks"], marker="o")   # line plot
plt.grid(True); plt.show()
df.plot(kind="bar", x="Name", y="Marks")       # pandas plotting
plt.show()
```

```
x = [1, 2, 3, 4]; y = [10, 20, 25, 30]
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1); plt.bar(["A", "B", "C", "D"], y, color="g")
plt.subplot(1, 2, 2); plt.pie(y, labels=["A", "B", "C", "D"], autopct=
plt.savefig("chart.png"); plt.show()
plt.hist([1, 2, 2, 3, 3, 3, 4, 4, 5], bins=5)     # histogram
```

Plot	Function	Used for
Line	plt.plot()	Trend over time
Scatter	plt.scatter()	Relation between two variables
Bar	plt.bar()	Compare categories
Histogram	plt.hist()	Distribution of data
Pie	plt.pie()	Percentage share

Important questions

* 1. Explain functional programming; map, filter, reduce with examples.
* 2. Explain JSON and XML handling in Python with programs.
* 3. Create NumPy arrays; explain ndim, shape, size, dtype, slicing and indexing.

* 4. What is a Pandas DataFrame? Explain loc and iloc, head(), data selection.
* 5. Write a program to draw a scatter plot using matplotlib.

6. Sample Experiments (24 - 30)

Exp 24: Check whether a JSON string contains a complex object

* A JSON value is COMPLEX if it is an object (dict) or array (list) - i.e. not a simple string / number / boolean / null.

```
import json
def is_complex(js):
    obj = json.loads(js)
    return isinstance(obj, (dict, list))
print(is_complex('{"a": 1}'))        # True
print(is_complex('[1, 2, 3]'))       # True
print(is_complex('"hello"'))         # False
print(is_complex('25'))              # False
# alternative: check any value inside is dict / list
```

Exp 25: NumPy array creation using array()

```
import numpy as np
a1 = np.array([1, 2, 3, 4, 5])                    # 1-D
a2 = np.array([[1, 2, 3], [4, 5, 6]])             # 2-D
a3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  # 3-D
a4 = np.array([1, 2, 3], dtype=float)
print(a1); print(a2); print(a3); print(a4)
```

Exp 26: ndim, shape, size, dtype

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print("ndim :", a.ndim)          # 2
print("shape:", a.shape)         # (2, 3)
print("size :", a.size)          # 6
print("dtype:", a.dtype)         # int64
```

Exp 27: Slicing, integer and Boolean indexing

```
a = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print(a[0:2, 1:])                # basic slicing [[20 30] [50 60]]
print(a[[0, 1, 2], [0, 1, 2]])  # integer indexing: diagonal [10 50 90
print(a[a > 50])                 # Boolean indexing [60 70 80 90]
```

Exp 28: Min, max, sum, cumulative sum

```
a = np.array([5, 3, 8, 1, 9, 2])
print("Min :", a.min())           # 1
print("Max :", a.max())           # 9
print("Sum :", a.sum())           # 28
print("Cumulative sum:", np.cumsum(a))   # [ 5  8 16 17 26 28]
```

Exp 29: Dictionary of lists -> DataFrame, explore

```
import pandas as pd
d = {"Name":   list("ABCDEFGHIJ"),
     "Age":    [20,21,19,22,20,23,21,22,19,20],
     "Marks":  [85,72,90,66,78,88,59,95,70,81],
     "Height": [160,172,165,170,158,180,175,169,162,177],
     "Weight": [55,68,60,65,52,75,70,62,58,72]}
df = pd.DataFrame(d)
print(df.head())                          # a) first 5 rows
print(df["Marks"])                        # b) select column
print(df[["Name", "Marks"]])
print(df.loc[2:4])                        # rows by label
print(df.iloc[0:3, 1:3])                  # by position
print(df[df["Marks"] > 80])               # condition
```

Exp 30: Scatter and plot of two columns

```
import matplotlib.pyplot as plt
plt.scatter(df["Height"], df["Weight"], color="blue")
plt.xlabel("Height (cm)"); plt.ylabel("Weight (kg)")
plt.title("Height vs Weight (scatter)")
plt.show()
df2 = df.sort_values("Height")
plt.plot(df2["Height"], df2["Weight"], marker="o")   # plot
plt.title("Height vs Weight (line)"); plt.show()
# observation: weight increases as height increases
# (positive correlation)
```

Revision: map / filter / reduce; json.loads / dumps; ElementTree; ndarray attributes ndim shape size dtype; loc (label) vs iloc (position); head(); plt.scatter / plot.